

Proyecto Python + MySQL

Formulário de Clientes con Tkinter y CRUD completo

Documento explicativo realizado el 24 de septiembre de 2026

Este proyecto es una aplicación de escritorio para administrar clientes. Usa una interfaz gráfica (Tkinter) para capturar datos, que se guardan en una base de datos MySQL mediante los **cuatro pasos básicos de cualquier sistema de datos**: crear, leer, actualizar y eliminar (CRUD).

1. Estructura del proyecto

```
python_mysql/
■■■ python_mysql.py          # Punto de entrada: interfaz grafica (Tkinter)
■■■ clientes.py              # Capa CRUD: operaciones sobre la tabla "usuarios"
■■■ connection.py           # Capa de conexion con MySQL
■■■ "Script de la base de datos.sql" # Instrucciones para crear la BD/insertar datos
```

Los tres archivos Python forman una arquitectura en **capas**. Cada archivo tiene una responsabilidad distinta y solo habla con el que está junto a él:

- **python_mysql.py** (interfaz) — construye la ventana y los botones. Le pide datos a **clientes.py** y le muestra el resultado.
- **clientes.py** (lógica de datos) — ejecuta las sentencias SQL de insertar, consultar, actualizar y borrar. Le pide la conexión a **connection.py**.
- **connection.py** (conexión) — solo sabe abrir una conexión a MySQL y devolverla.

Tu (usuario)	Interfaz (Tkinter)	CRUD (clientes.py)	Conexion (connection.py)
escribe datos -->	python_mysql.py ----->	CClientes ----->	CConnection ----->
<-- ve la tabla ---	<----- resultados -----	<-----	<-----

Ventaja de separar en capas: si mañana se cambia Tkinter por otra interfaz, solo se tocaría `python_mysql.py`; si cambia la base de datos (por ejemplo a PostgreSQL), solo se tocaría `connection.py`.

2. La base de datos (MySQL)

El archivo *Script de la base de datos.sql* crea la base de datos y la tabla, y deja algunos ejemplos. La estructura es la siguiente:

```
CREATE DATABASE clientesdb;          -- crea la base de datos
USE clientesdb;                      -- la selecciona como activa
CREATE TABLE usuarios (
    id          INT AUTO_INCREMENT PRIMARY KEY NOT NULL, -- clave (se llena sola)
    nombres     VARCHAR(50),                -- texto de hasta 50 caracteres
    apellidos   VARCHAR(50),
    sexo        VARCHAR(20)
);
```

Tabla **usuarios** (4 columnas):

id	nombres	apellidos	sexo
1	Erik	Jarquín	Masculino

EJEMPLO DE FILA

```
id      = 1 (auto_increment)
nombres = 'Erik'
sexo    = 'Masculino'
```

auto_increment es la clave: MySQL asigna el número 1, 2, 3... automáticamente, por eso al insertar se manda *null* en esa posición y la tabla se llena sola. Ese id es lo que permite seleccionar 'cuál cliente' se actualiza o se borra.

3. Cómo funciona el CRUD (el corazón del proyecto)

CRUD son las siglas en inglés de las 4 operaciones que hace cualquier sistema de datos. Aquí se reparten entre las funciones de **clientes.py** y los botones de la interfaz:

OPERACION	SQL que ejecuta	FUNCION en clientes.py
CREAR	INSERT INTO usuarios VALUES (null,?,?,?)	ingresarClientes()
LEER	SELECT * FROM usuarios	mostrarClientes()
ACTUALIZAR	UPDATE usuarios SET ... WHERE id=?	modificarClientes()
ELIMINAR	DELETE FROM usuarios WHERE id=?	eliminarClientes()

3.1 Leer (mostrar los clientes)

Cadena completa desde la interfaz:

```
# Boton "Guardar/Actualizar" presionado? No -> se pinta la tabla.
# python_mysql.py
datos = CClientes.mostrarClientes() # pide la lista
for row in datos:                   # y la agrega fila por fila
    tree.insert("", "end", values=row)

# clientes.py
cursor.execute("select * from usuarios;")
miResultado = cursor.fetchall()     # lista de tuplas: [(1, 'Erik', ...), ...]
```

3.2 Crear (guardar un cliente nuevo)

```
# Boton "Guardar" -> guardarRegistros()
nombres = textBoxNombres.get()      # lee el Entry de la interfaz
apellidos = textBoxApellidos.get()
sexo = combo.get()
CClientes.ingresarClientes(nombres, apellidos, sexo) # INSERT

# clientes.py
sql = "insert into usuarios values (null,%s,%s,%s);"
valores = (nombres, apellidos, sexo) # una TUPLA, en el mismo orden
cursor.execute(sql, valores)         # los %s se sustituyen aqui
cone.commit()                       # guarda el cambio de forma definitiva
```

3.3 Actualizar (modificar un cliente)

```
# Boton "Actualizar" -> modificarRegistros(). Necesita saber CUAL fila.
CClientes.modificarClientes(id_usuario, nombres, apellidos, sexo) # UPDATE

# clientes.py
sql = "update usuarios set nombres=%s, apellidos=%s, sexo=%s where id=%s;"
valores = (nombres, apellidos, sexo, id)
cursor.execute(sql, valores)         # WHERE filtra: solo cambia ese id
```

3.4 Eliminar (borrar un cliente)

```
# Boton "Eliminar" -> eliminarRegistros(). Solo necesita el id.
id_usuario = textBoxId.get()
CClientes.eliminarClientes(id_usuario) # DELETE

# clientes.py
sql = "delete from usuarios where id=%s;"
valores = (id,)                     # TUPLA de un solo elemento: la coma es obligatoria
cursor.execute(sql, valores)
cone.commit()
```

Por qué se usa %s y no se pega el texto en el SQL: por seguridad. Separar la consulta (sql) de sus valores (valores) evita la 'inyección SQL' y errores cuando el texto lleve comillas o apóstrofes.

4. Seguridad y buenas prácticas ya aplicadas

- **Consultas parametrizadas (%s).** Los datos del usuario nunca se pegan directamente en la sentencia SQL.
- **Comillas simples para valores SQL.** Las tuplas con un solo valor llevan coma (id,).
- **commit() tras cambiar datos.** Sin él, MySQL no guarda los INSERT/UPDATE/DELETE.
- **Cierre de conexiones.** Cada método abre su conexión y la cierra al terminar (evita conexiones colgadas).

5. Errores corregidos (24/09/2026)

Los 4 errores reales detectados en la primera revisión fueron corregidos en el código y comentados. Estado actual:

- **connection.py** — se eliminó la llamada `connectionBaseDeDatos()` que se ejecutaba al importar y abría una conexión 'fantasma' (podía romper el programa si MySQL estaba apagado). **CORREGIDO.**
- **connection.py** — el bloque `except` ya no devuelve `return conexion` (variable inexistente = `UnboundLocalError`), ahora devuelve `None`. **CORREGIDO.**
- **clientes.py** — ahora importa `import mysql.connector` de forma directa en lugar de depender de `connection.py`. **CORREGIDO.**
- **python_mysql.py** — los 4 Label usan la fuente correcta `("arial", 12)` en vez de `("arial,12")`. **CORREGIDO.**
- **Extras:** los métodos de `clientes.py` ahora detectan `None` y cancelan la operación con un aviso claro en consola (antes el programa reventaba si no había MySQL), y el arranque quedó protegido con `if __name__ == "__main__":`. **CORREGIDO.**

Mejoras pendientes opcionales (no rompen nada, pero mejoran el código)

- Guardar contraseña y datos de conexión en variables de entorno (env) en vez de escribirlos en el código.
- Usar **try/finally** (o un gestor de contexto) para garantizar que la conexión se cierre aunque algo falle.
- En `actualizarTreeview()`, llamar a `mostrarClientes()` una sola vez y guardarlo en 'datos' (hoy se llama 2 veces).
- Eliminar el `commit()` después del `SELECT` (no es necesario) y los `print()` de depuración del módulo de conexión.
- Eliminar la clase 'fantasma' `FormularioClientes` y el archivo `get-pip.py` que quedó en la carpeta raíz.
- Agregar confirmación (`messagebox.askyesno`) antes de borrar un registro.

6. Cómo ejecutar el proyecto

```
# 1) Requisitos previos
#   - Python 3 instalado
#   - MySQL instalado y corriendo en el puerto 3306

# 2) Instalar la libreria de conexion
pip install mysql-connector-python

# 3) Crear la base de datos: ejecutar el archivo
#   "Script de la base de datos.sql" dentro de MySQL Workbench o phpMyAdmin

# 4) Verificar el acceso en connection.py
#   user="root", password="tu_pass", database="clientesdb"

# 5) Ejecutar el programa
python python_mysql.py
```

Al arrancar se abre una ventana. En la parte izquierda se escriben los datos (el **id** solo hace falta para Actualizar/Eliminar), a la derecha se ve la tabla. Al hacer clic en una fila, sus datos se copian automáticamente a los cuadros de texto.

7. Glosario rápido

Término	Qué significa aquí
Cursor	Objeto de Python que ejecuta una sentencia SQL y guarda el resultado.
Tupla	Agrupación de valores entre paréntesis: (a, b, c). Inmutable. Con un solo valor lleva coma: (id,).
fetchall()	Trae todas las filas del resultado como una lista de tuplas.
commit()	Confirma de forma definitiva los cambios (INSERT/UPDATE/DELETE).
rowcount	Cantidad de filas afectadas por la última consulta (ej. '1 Registro ingresado').
Entry	Cuadro de texto de una sola línea en la interfaz.
Combobox	Lista desplegable para elegir un valor (aquí, el sexo).
Treeview	El 'grid' de la interfaz donde se muestran los registros.
StringVar	Variable de Tkinter conectada a un widget; con .set() cambia su valor.
mainloop()	Arranca la ventana y mantiene la aplicación viva atendiendo eventos.
Inyección SQL	Ataque que se evita al separar la consulta de sus valores (%s).

Resumen final: la interfaz captura datos, clientes.py los convierte en sentencias SQL seguras, y connection.py abre la puerta a MySQL. Cada botón de la ventana es una de las 4 letras del CRUD.